

PROGRAM-1

AIM:

Python program to display details about the operating system, working directory, files and directories in the current directory, lists the files and all directories, scan and classify them as directories and files.

CODE:

```
import os
import platform
import socket

def display_os_details():
    print("Operating System:", platform.system(), platform.release())
    print("Version:", platform.version())
    print("Platform:", platform.platform())
    print("Processor:", platform.processor())
    print("Device Name:", socket.gethostname())
    print("System Type:", platform.architecture()[0])

def display_working_directory():
    cwd = os.getcwd()
    print("\nCurrent Working Directory:", cwd)

def classify_files_and_directories():
    print("\nFiles and Directories in the Current Directory:")
    items = os.listdir()
    files = []
    directories = []

    for item in items:
        if os.path.isfile(item):
            files.append(item)
        elif os.path.isdir(item):
            directories.append(item)

    print("\nDirectories:")
    for directory in directories:
        print("-", directory)

    print("\nFiles:")
    for file in files:
        print("-", file)

display_os_details()
display_working_directory()
classify_files_and_directories()
```

PROGRAM-2

AIM: Python program to convert an array to an array of machine values and vice versa.

CODE:

```
import struct

original_array = [1.5, 2.3, 3.7, 4.1]
print("Original Array:", original_array)

machine_values = b''.join(struct.pack('f', value) for value in original_array)
print("Machine Values (bytes):", machine_values)

converted_array = [struct.unpack('f', machine_values[i:i+4])[0] for i in range(0,
len(machine_values), 4)]
print("Converted Array:", converted_array)
```

PROGRAM-3

AIM: Python program to get information about the file pertaining to the file mode and to get time values with components using local time and gm time.

CODE:

```
import os
import time

file_path = 'C:/Users/Parth/OneDrive/Desktop/Python.txt'

file_stat = os.stat(file_path)
file_mode = oct(file_stat.st_mode)[-3:]
local_time = time.localtime(file_stat.st_mtime)
gm_time = time.gmtime(file_stat.st_mtime)

local_time_str = time.strftime('%Y-%m-%d %H:%M:%S', local_time)
gm_time_str = time.strftime('%Y-%m-%d %H:%M:%S', gm_time)

print("File Mode:", file_mode)
print("Last Modified (Local Time):", local_time_str)
print("Last Modified (GMT):", gm_time_str)
```

PROGRAM-4

AIM: Python program to connect to Google using socket programming.

CODE:

```
import socket

def connect_to_google():
    host = 'www.google.com'
    port = 80
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((host, port))
        request = f'GET / HTTP/1.1\r\nHost: {host}\r\nConnection: close\r\n\r\n'
        s.sendall(request.encode())
        response = b''
        while True:
            chunk = s.recv(4096)
            if not chunk:
                break
            response += chunk
        print(response.decode()[:500]) # Print only the first 500 characters

if __name__ == "__main__":
    connect_to_google()
```

PROGRAM-5

AIM: Python program to perform Array operations using Numpy package.

CODE:

```
import numpy as np

array_1d = np.array([1, 2, 3, 4, 5])
array_2d = np.array([[1, 2, 3], [4, 5, 6]])

print("1D Array:", array_1d)
print("2D Array:\n", array_2d)

print("Shape of 1D Array:", array_1d.shape)
print("Shape of 2D Array:", array_2d.shape)

array_added = array_1d + 10
array_multiplied = array_1d * 2

print("Array after adding 10:", array_added)
print("Array after multiplying by 2:", array_multiplied)

print("Element at index 2 in 1D Array:", array_1d[2])
print("Slicing 2D Array (first row):", array_2d[0, :])

reshaped_array = array_1d.reshape(5, 1)
print("Reshaped 1D Array to 2D:\n", reshaped_array)

array_concatenated = np.concatenate((array_1d, array_added))
print("Concatenated Array:", array_concatenated)

mean_value = np.mean(array_1d)
std_value = np.std(array_1d)

print("Mean of 1D Array:", mean_value)
print("Standard Deviation of 1D Array:", std_value)
```

```
np.save('my_array.npy', array_1d)
loaded_array = np.load('my_array.npy')

print("Loaded Array from file:", loaded_array)

boolean_indexing = array_1d[array_1d > 2]
print("Elements greater than 2:", boolean_indexing)
```

PROGRAM-6

AIM: Python program to perform Data Manipulation operations using Pandas package.

CODE:

```
import pandas as pd

data = {
    'Name': ['Chehak', 'Parth', 'Kunal', 'Ankush', 'Deepali'],
    'Age': [19, 20, 22, 24, 21],
    'City': ['Gurgaon', 'Delhi', 'Faridabad', 'Chandigarh', 'Agra'],
    'Salary': [90000, 80000, 65000, 90000, 85000]
}

df = pd.DataFrame(data)

print("Original Data:")
print(df)

print("\nFiltered Data (Age > 25):")
filtered_df = df[df['Age'] > 25]
print(filtered_df)

print("\nData Sorted by Salary (Descending):")
sorted_df = df.sort_values(by='Salary', ascending=False)
print(sorted_df)

df['Salary with Bonus'] = df['Salary'] * 1.10
print("\nData with 'Salary with Bonus' column added:")
print(df)

print("\nAverage Salary by City:")
grouped_df = df.groupby('City')['Salary'].mean()
print(grouped_df)

print("\nStatistics for Age column:")
print(df['Age'].agg(['mean', 'min', 'max']))

df = df.drop(columns=['City'])
print("\nData after dropping 'City' column:")
print(df)
```

PROGRAM-7

AIM: Python program to display multiple types of charts using Matplotlib package.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(r"C:/Users/Chehak/OneDrive/Desktop/Practical 7.csv")
print(df)

categories = df['Category']
values = df['Value']
x_values = df['X']
y_values = df['Y']

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.plot(x_values, y_values, marker='o', linestyle='-', color='b')
plt.title('Line Chart')
plt.xlabel('X Values')
plt.ylabel('Y Values')

plt.subplot(2, 2, 2)
plt.bar(categories, values, color='g')
plt.title('Bar Chart')
plt.xlabel('Categories')
plt.ylabel('Values')

plt.subplot(2, 2, 3)
plt.scatter(x_values, y_values, color='r')
plt.title('Scatter Plot')
plt.xlabel('X Values')
plt.ylabel('Y Values')

plt.subplot(2, 2, 4)
plt.pie(values, labels=categories, autopct="%1.1f%%", startangle=90)
plt.title('Pie Chart')
```

```
plt.tight_layout()  
plt.show()
```

PROGRAM-8

AIM: Python program to perform File Operation on Excel Data Set

CODE:

```
import pandas as pd
excel_filename=r"C:\Users\ChehakloneDrive\ドキュメント\VP\Practical8.xlsx"
df = pd.read_excel(excel_filename)
print("Original Data:")
print(df)
df [ 'Age'] = df['Age'] + 1
print("\nData after increasing Age by 1:")
print(df)
df[ 'Status'] = df[ 'Age']-apply(lambda x: 'Adult' if x >= 18 else 'Minor')
print("\nData with new Status column:")
print(df)
df_sorted = df.sort_values (by='Age', ascending=False)
print("\nData sorted by Age in descending order:")
print(df_sorted)
modified_filename = "extended modified_data.xlsx"
df_sorted.to_excel(modified_filename, index=False)
print(f"\nModified data saved to (modified_filename)")
```

PROGRAM-9

AIM Python program to implement with Python Sci Kit-Learn & NLTK.

CODE:

```
import nltk
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn import metrics

texts = [
    "I love programming",
    "Python is amazing",
    "I hate bugs",
    "Debugging is fun",
    "I enjoy learning new languages"
]
labels = [1, 1, 0, 1, 1]

nltk.download('stopwords')
stop_words = nltk.corpus.stopwords.words('english')

vectorizer = TfidfVectorizer(stop_words=stop_words)
X = vectorizer.fit_transform(texts)

X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2,
random_state=42)

classifier = MultinomialNB().fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

PROGRAM-10

AIM: Python program to implement with Python NLTK/Spicy/Py NLPI.
CODE:

```
download('punkt')
nltk.download('stopwords')
nltk.download('maxent_ne_chunker')
nltk.download('words')
nltk.download('averaged_perceptron_tagger')
import nltk from nltk.tokenize import word_tokenize from nltk.corpus import
stopwords from nltk import ne_chunk, pos_tag
from nltk.tree import Tree
text = "Apple is looking to buy a startup in the UK for $1 billion."
nltk_tokens = word_tokenize(text)
print("NLTK Tokens:", nltk_tokens)
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in nltk_tokens if word.lower() not in
stop_words]
print("InFiltered Tokens (No Stopwords):", filtered_tokens)
pos_tags = pos_tag(filtered_tokens)
print("\nPOS Tags:", pos_tags)
named_entities = ne_chunk(pos_tags)
print("\nNamed Entities (NER) from NLTK:") for subtree in named_entities:
if isinstance(subtree, Tree):
entity = ""-join([word for word, tag in subtree.leaves()])
print(entity + " - " + subtree.label())
```